

강병훈 (Byunghoon Kang)

Frontend Engineer

Contact: byunghoon.kang.job@gmail.com

Github: <https://github.com/KBH-js>

LinkedIn: <https://www.linkedin.com/in/byunghoon-kang-293369302/>

Summary

Module Federation 기반 아키텍처 설계와 엔지니어링 프로세스 자동화를 통해 제품의 안정성을 확보하고 팀의 개발자 경험(DX)을 고도화하는 5년 차 프론트엔드 엔지니어입니다.

마이크로 프론트엔드 웹 데스크톱 환경에서 10개 이상의 앱을 독립 배포 구조로 확립하고 런타임에 통합한 경험이 있습니다. 또한, AI 에이전트를 활용한 고속 개발 패러다임에 맞추어 CI 가드레일 등 검증 파이프라인을 고도화하여 AI 생성 코드의 결함을 사전에 차단하고 코드 리뷰 공수를 줄여 안정성을 확보하는 데 강점이 있습니다.

기술 스택

- **Languages & Core:** TypeScript, JavaScript, HTML, CSS
- **Libraries & State:** React 19, TanStack Query, Zustand
- **Architecture & Build:** Micro Frontends(Module Federation), Rspack, pnpm Monorepo
- **Testing & CI/CD:** Vitest, Playwright, MSW, Storybook, GitHub Actions
- **AI & DX Tooling:** Claude Code, Cursor, MCP, AGENTS.md Specification

Key Project

[웹 기반의 데스크톱 환경 구축](#)(at 다키클라우드 코리아) | 2025.08 ~ 현재

Challenge: 웹 환경에서 데스크톱 OS 수준의 멀티태스킹 UI를 제공하고, 다수의 독립된 앱을 호스트 리프레시 없이 런타임에 동적으로 로드할 수 있는 아키텍처 환경이 요구되었습니다.

Solution:

- **Module Federation** 아키텍처 설계
각 마이크로 앱의 독립 배포 보장을 위해 런타임 동적 Remote URL 주입 구조를 구현하고, 앱 단위 공유 의존성을 최적화했습니다.

- **Windowing** 레이어 인터페이스 구현
React-rnd 라이브러리를 활용하여 웹 내에서 창 이동, 리사이징, 멀티태스킹 레이어 인터페이스를 구축했습니다.
- **AI** 워크플로우를 통한 **PoC** 가속
요구사항 정의 단계부터 **AI** 에이전트를 활용하여 핵심 인터페이스의 보일러플레이트 코드 생성을 자동화하였으며, 2주 내 프로토타입 빌드를 완료하여 아키텍처 타당성을 검증했습니다.

Result:

- 10개 이상의 독립된 마이크로 앱이 단일 데스크톱 셀에서 통합 및 배포되는 구조를 확립했습니다.
- 사내 기술 PoC 및 데모 시연을 성공적으로 완료한 후 기능을 고도화했으며, 현재는 실서비스 배포 전 QA 단계를 수행 중입니다.

경력 사항

다키클라우드 코리아 | *Frontend Engineer* | 2025.07 ~ 현재

- 마이크로 프론트엔드 아키텍처 수립 및 배포 독립성 확보
Host와 Remote 애플리케이션 간의 의존성 결합도를 낮추는 동적 **Module Federation** 구조를 설계하여, 개별 마이크로 앱 단위의 독립적인 배포 사이클을 확립했습니다.
- 클라우드 관리 플랫폼 구현
OpenStack 기반 대시보드 내 네트워크 가상화 모듈을 담당했습니다. 다양한 인프라 리소스가 연계된 복잡한 네트워킹 모델의 **REST API**를 연동하고, 사용자 친화적인 **UI**로 구현했습니다.
- **React Compiler** 마이그레이션
점진적 도입 전략을 통해 모노레포 내 전체 앱의 마이그레이션을 진행했으며, 불필요해진 수동 메모이제이션을 **Codemod**로 안전하게 제거했습니다. 이 과정에서 **react-hook-form watch()**의 렌더링 참조가 컴파일러 캐싱과 충돌해 화면이 갱신되지 않는 현상을 발견하고, **useWatch** 구조로 전환하여 해결했습니다.
- 테스트 인프라 구축
Vitest/MSW 통합 테스트와 **Playwright E2E** 테스트 인프라를 설계하고 초기 검증을 위한 160여 개의 테스트 파일을 작성했습니다. CI 실행 속도 최적화를 위해 변경 파일만 선별 실행하는 **PR** 게이트를 도입하고, 전체 테스트 세트는 **Nightly** 스케줄러로 구동했습니다.
- 디자인 시스템 토큰 파이프라인 및 시각 회귀 테스트(**VRT**) 구축
 - 공용 디자인 시스템의 **3-layer** 디자인 토큰 파이프라인을 운영하고, 토큰 드리프트 및 하드코딩된 **raw** 컬러 사용을 차단하는 **CI** 가드레일을 확립했습니다.
 - **Playwright** 기반 시각 회귀 테스트를 도입해 **light/dark** 테마 매트릭스 베이스라인 300여 장을 운영했으며, 픽셀 단위 비교 검증을 통해 의도하지 않은 **UI** 변경 요소를 사전에 차단했습니다.
- **DX** 최적화 및 협업 프로세스 개선
 - 디자이너가 코드를 직접 수정하는 워크플로우에서 발생하기 쉬운 반복적 실수를 차단하기 위해, **CI** 내 검증 스크립트 및 코드 리뷰 **Skill**을 만들어 리뷰 공수를 절감했습니다.
 - 팀 개발 규약을 **AGENTS.md**로 단일화하고 **Claude Code hook** 및 **Lint** 룰로 강제화하여, **AI** 에이전트가 생성하는 코드가 팀 컨벤션을 엄격히 준수하도록 프로세스를 고도화했습니다.

티맥스 **ANC** | *Frontend Engineer* | 2021.04 – 2024.11

- **B2B/공공 SI** 대응을 위한 공통 스토어 라이브러리 개선 및 운영
 - 서울시교육청, 푸디스트 등 **SI** 사업에서 공통 기능 및 **MobX** 기반 전역 상태 관리 라이브러리를 고도화하고 유지보수했습니다.
 - 웹뷰 기반 모바일 서비스 구현 시 네이티브-웹 간 통신 규격을 준수하여, 데스크톱과 모바일 환경 전반에서 일관된 반응형 사용자 경험을 제공했습니다.
 - 브라우저 렌더링 성능 최적화
10,000건 이상의 대용량 조직도 데이터 처리 시 발생하는 **DOM** 병목을 해결하기 위해 가상 스크롤 기법을 도입했습니다. 또한, 실시간 검색 기능에 디바운싱을 적용하고, 클라이언트 사이드 캐싱을 구현하여 데이터 재진입 시 즉각적인 **UI** 응답성을 확보했습니다.
-

학력

KAIST (한국과학기술원) | 수리과학과 학사 | 2012.02 – 2020.02 (학위 기간 중 군 복무 포함)

언어

- 영어: 전문적인 업무 구사 가능 (**Professional Working Proficiency**)
 - TOEFL iBT 104
 - OPIc IH
 - 공군작전사령부 어학병 근무로 병역의무 수행